

Projeto de Conclusão de Bootcamp

Dio.me e Accenture

NicoAI – Assistente Virtual Didático de Finanças Pessoais

DOCUMENTAÇÃO DE REQUISITOS DE SOFTWARE

Stephanie Thawane Cavalcante Madia

SUMÁRIO

CAPÍTULO 1 – Introdução

1.2	Objetivo
1.2	Escopo
1.3	Definições, Siglas e Abreviações
1.4	Referências
1.5	Informações Adicionais
1.5.1	Dados da Instituição
1.5.2	Descrição da Empresa, Histórico, Ramo de Atividade.....
1.5.3	Legislação de Software
1.6	Visão Geral

CAPÍTULO 2 – Descrição Geral do Produto

2.1	Funções do Produto.....
2.2	Característica do Usuário.....
2.3	Limites, Suposições e Dependências
2.4	Requisitos Adiados.....

CAPÍTULO 3 – Requisitos Específicos

3.1	Diagrama de Casos de Uso
3.1.1	Especificações de Casos de Usos
3.2.1	Interfaces do Usuário
3.2.2	Interfaces de Software.....
3.2.3	Interfaces de Sistema
3.2.4	Interfaces de Hardware
3.2.5	Interfaces de Comunicação
3.3	Outros Requisitos
3.4	Especificação de Requisitos de Dados.....
3.4.1	Exemplo de Injeção de Contexto.....

CAPÍTULO 4 – PLANO DE TESTES E CRITÉRIOS DE HOMOLOGAÇÃO

4.1	Avaliação e Métricas de Qualidade.....
4.2	Cenários de Teste Estruturados.....
4.3	Resultados e Relatório de Conclusões.....
4.4	Métricas Avançadas de Observabilidade.....

CAPÍTULO 5 – Projeto de Software

5.1	Estrutura do Repositório de Scripts.....
5.2	Fluxo de Carga em Memória.....

APÊNDICE 1 – PROTÓTIPOS E RELATÓRIOS DE ANÁLISE

ANEXO 1 – REFERÊNCIAS

1.1 Objetivo

Este documento tem por objetivo apresentar os requisitos funcionais e não funcionais que o assistente virtual NicoAI deve atender. A especificação detalhada do comportamento do software em desenvolvimento para fins de avaliação pelos profissionais da plataforma e conclusão do curso de capacitação tecnológica em análise de dados.

1.2 Escopo

O software será nomeado como NicoAI – Agente de Inteligência Artificial para Educação Financeira, um sistema que visa ajudar pessoas a compreenderem sua realidade financeira e organizarem seus orçamentos pessoais, com o objetivo de ser eficiente para a interpretação didática de transações, análise de perfil e consumo, desde a leitura de bases históricas, categorização de despesas e geração de insights em linguagem natural, mitigando a complexidade e a falta de clareza na gestão de finanças.

Esta aplicação deverá ter funcionalidades básicas de gerenciar dados de perfil e gerenciar repositórios de transações que darão suporte para as funcionalidades fundamentais e mais importantes para o cliente e que serão descritas a seguir.

O sistema deverá oferecer as seguintes funções que são fundamentais para o cliente: carregar perfil do investidor, analisar transações recentes, consultar atendimentos anteriores, avaliar produtos disponíveis, responder consultas didáticas e filtrar perguntas fora de escopo. Sabe-se que um usuário está associado a nenhum ou a um único perfil de investidor, o qual pode incluir, no máximo, um objetivo principal, um valor de patrimônio total e uma reserva de emergência atual.

O usuário também pode possuir um histórico contendo uma ou mais transações financeiras, divididas entre colunas de data, categoria, descrição e valor. Além disso, o usuário pode estar associado a um histórico de atendimentos anteriores contendo interações prévias com a inteligência artificial. Em relação aos produtos disponíveis, o sistema pode listar um ou mais ativos do mercado, contendo tipo de investimento, rentabilidade estimada e nível de risco associado. Ressalta-se ainda que é possível que um usuário faça perguntas conceituais de finanças sem, necessariamente, possuir transações registradas em seu nome no arquivo local.

O NicoAI irá operar no conceito de Geração Aumentada de Recuperação (RAG), o sistema deverá ser capaz de injetar dados locais estruturados diretamente no prompt do modelo de linguagem de grande porte (LLM). As bases de dados serão identificadas por formatos específicos, sendo o perfil do investidor e o catálogo de produtos armazenados em formato JSON, enquanto as movimentações financeiras e os históricos de chat serão mantidos em formato CSV. Dessa forma, quando for realizada uma consulta pelo usuário, o interpretador local deverá varrer esses arquivos para construir a string de contexto que alimentará o motor de inteligência artificial de forma integrada.

Para efetuar a montagem do contexto do usuário, deverão ser extraídos e concatenados: o nome do cliente, idade, perfil de investidor (Conservador, Moderado ou Arrojado), objetivo principal, valor do patrimônio total, reserva de emergência atual, a listagem completa das transações recentes com seus respectivos valores, os registros de conversas anteriores e a estrutura detalhada de produtos financeiros mapeados no ecossistema local.

Caso o arquivo de transações esteja vazio, este deverá manter no mínimo o cabeçalho estruturado e o sistema utilizará apenas os dados gerais de perfil para gerar exemplos conceituais de finanças.

O usuário, caso queira tirar dúvidas ou analisar suas finanças, utilizará uma interface de chat unificada (Streamlit), que mostrará o histórico de mensagens em tempo real e processará os textos enviados. O agente analisará a pergunta utilizando regras de prompt rígidas, gerando

respostas que devem ser estritamente sucintas, diretas e limitadas a no máximo 3 parágrafos, explicando conceitos de maneira amigável, como se estivesse conversando com um amigo, e sempre terminando a interação perguntando se o cliente conseguiu compreender a explicação técnica fornecida.

Ao receber um prompt do usuário, o mecanismo de validação avaliará se a pergunta está correlacionada ao tema de finanças pessoais e educação financeira. Caso ocorra uma tentativa de consultar temas fora do escopo, o sistema ativará uma rotina automática de rejeição, recusando a resposta educadamente e lembrando o usuário sobre o seu papel exclusivo de educador financeiro, bloqueando a geração de conteúdos não pertinentes.

Para a aplicação de diretrizes de segurança sobre investimentos, o sistema notificará uma mensagem educativa e jamais fará recomendações de compra ou venda de ativos específicos. O agente está programado para apenas explicar o funcionamento dos mecanismos do mercado e ensinar como os produtos financeiros operam teoricamente, garantindo neutralidade pedagógica e proteção ao usuário.

O sistema mostrará na interface de conversação o resultado imediato das interações e dúvidas textuais enviadas, mantendo o fluxo histórico visível na tela para que o usuário possa acompanhar o progresso de sua consulta atual de forma fluida. Devido à premissa de execução estritamente local e sem persistência em bancos relacionais complexos, as respostas geradas são renderizadas diretamente na interface de forma assíncrona, respondendo às demandas imediatas do usuário em tempo real.

O sistema NicoAI oferecerá benefícios significativos, reduzindo o analfabetismo financeiro e eliminando a dependência de plataformas de nuvem pagas ou inseguras, tornando a educação financeira mais acessível, privada e estruturada. Este agilizará a compreensão de orçamentos e facilitará a identificação de hábitos de consumo ruins, garantindo maior eficiência no controle financeiro pessoal dos cidadãos. Com funcionalidades automatizadas, como a montagem de contexto via RAG, filtros rigorosos de escopo e análise segura de dados locais, o sistema trará mais transparência, permitindo uma tomada de decisão consciente e um melhor aproveitamento dos recursos disponíveis no presente e no futuro.

1.3 Definições, Siglas e Abreviações - Depois verificar aqui

<i>Sigla ou Abreviação</i>	<i>Definição</i>
<i>Cód.</i>	Abreviação para a palavra código.
<i>Apd.</i>	Abreviação para apêndice
<i>RF_B</i>	Abreviação para referências básicas
<i>RF_F</i>	Abreviação para referências fundamentais

1.4 Referências

Nº	Título	Data Aquisição	Responsável pelo Fornecimento
1	Foi utilizado um link de repositório como referência para o desenvolvimento.	20/06/2026	Venilton Falvo Jr. - Dio.me
2	Foi utilizado um módulo de vídeos da Dio.me como referência para o desenvolvimento.	20/06/2026	Venilton Falvo Jr. - Dio.me

1.5 Informações Adicionais

1.5.1 Dados da Instituição

As empresas que irão dar suporte na realização deste software são a Dio.me e também a Accenture, o desenvolvimento deste sistema será feito por uma aluna do bootcamp de Análise de Dados da Accenture.

1.5.2 Dados da Empresa

A Dio.me é uma plataforma de educação tecnológica focada em desenvolvimento de software e inovação, que atua na capacitação de talentos por meio de bootcamps, cursos abertos e conexão de profissionais com o mercado de trabalho.

A Accenture é uma empresa global de consultoria de gestão e tecnologia que transforma negócios através do uso estratégico de Data & AI (Dados e Inteligência Artificial), engenharia de dados, business intelligence e soluções em nuvem. A empresa atua desenhando arquiteturas de dados complexas e gerando insights analíticos para direcionar a tomada de decisões de grandes indústrias globalmente.

1.5.3 Legislação de Software

Ambas empresas e visitantes terão direito ao software. O software será de uso livre e de cópia, conforme a legislação estipulada. Além disso, os dados contidos nele serão tratados conforme as diretrizes da Lei Geral de Proteção de Dados Pessoais (LGPD). Como o sistema possui fins estritamente acadêmicos e de demonstração, serão utilizados dados fictícios para quaisquer simulações de interações e armazenamento, garantindo que nenhuma informação pessoal identificável real seja exposta, coletada ou processada de forma indevida.

1.6 Visão Geral

Este documento está estruturado de maneira clara e objetiva em capítulos sequenciais, tópicos especializados, apêndices e anexos, organizados de forma a guiar o leitor desde os conceitos de negócio até os detalhes da implementação técnica.

O Capítulo 2 fornece uma descrição geral do produto e do ecossistema da aplicação. Apresenta uma síntese de alto nível das funções que o sistema atende, as características dos usuários mapeados e as limitações técnicas e dependências de hardware e ambiente locais necessárias para a execução estável do software.

No Capítulo 3, os requisitos são detalhados a nível de análise técnica e desenvolvimento. Esta seção contém o Diagrama de Casos de Uso do agente e suas respectivas especificações de fluxo básico, o detalhamento das interfaces externas do usuário (UI) construídas sob o framework Streamlit, a arquitetura de comunicação interna e a Especificação de Requisitos de Dados baseada em arquivos estruturados (CSV e JSON).

No Capítulo 4, é apresentado o Plano de Testes e Critérios de Homologação. São descritos os cenários de teste estruturados para validação do comportamento da inteligência artificial generativa, os parâmetros de assertividade de contexto e o relatório descritivo de conclusões sobre o desempenho do modelo local (Ollama).

O Capítulo 5 detalha o Projeto de Software através do mapeamento arquitetural do repositório, descrevendo a árvore de diretórios e a organização dos scripts em Python que estruturam a lógica de negócio do agente.

Na sequência, o Apêndice A engloba o roteiro de apresentação e o modelo de Pitch Comercial estruturado para a demonstração prática da solução técnica desenvolvida.

Por fim, os Anexos listam as referências bibliográficas, documentações oficiais e repositórios de código que fundamentaram o levantamento de requisitos e o desenvolvimento do ecossistema do assistente virtual.

2.1 Funções do Produto

Referência	Função	Visibilidade	Atributo	Detalhe	Categoria
RF_B1	Gerenciar Dados de Perfil	Evidente	-	-	-
RF_B2	Gerenciar Repositório de Transações	Evidente	-	-	-
RF_F1	Carregar Perfil do Investidor	Oculto	Consistência	Leitura automatizada do arquivo perfil_investidor.json	Obrigatório
RF_F2	Analisar Transações Recentes	Evidente	Tolerância a Falhas	Suportar arquivo transacoes.csv vazio mantendo a estrutura mínima	Obrigatório
RF_F3	Responder Consultas Didáticas	Evidente	Tempo de resposta	Máximo 10 segundos	Desejável
RF_F4	Filtrar Perguntas Fora de Escopo	Evidente	Segurança	Identificar e rejeitar perguntas não financeiras com mensagem padrão	Obrigatório
RF_S1	Exibir Linha do Tempo do Chat	Evidente	Usabilidade	Renderização assíncrona das mensagens via interface Streamlit	Obrigatório

2.2 Características do Usuário

Os usuários deste sistema são os próprios estudantes, desenvolvedores ou avaliadores do bootcamp da DIO.me. Por se tratar de um assistente virtual integrado a uma interface de chat direta, o sistema exige apenas o nível de conhecimento básico em informática (saber digitar e ler mensagens em ambiente web).

Como a interface é intuitiva e baseada em conversação natural, não há necessidade de treinamentos ou capacitação técnica prévia para a utilização da ferramenta por parte do usuário final.

Todavia, caso um desenvolvedor necessite interagir com o código-fonte, realizar modificações, utilizá-lo como inspiração ou compreender o ecossistema mais a fundo, este deverá consultar o arquivo de documentação técnica inicial (README.md) para orientações sobre a configuração do ambiente e execução dos modelos.

2.3 Limites, Suposições e Dependências

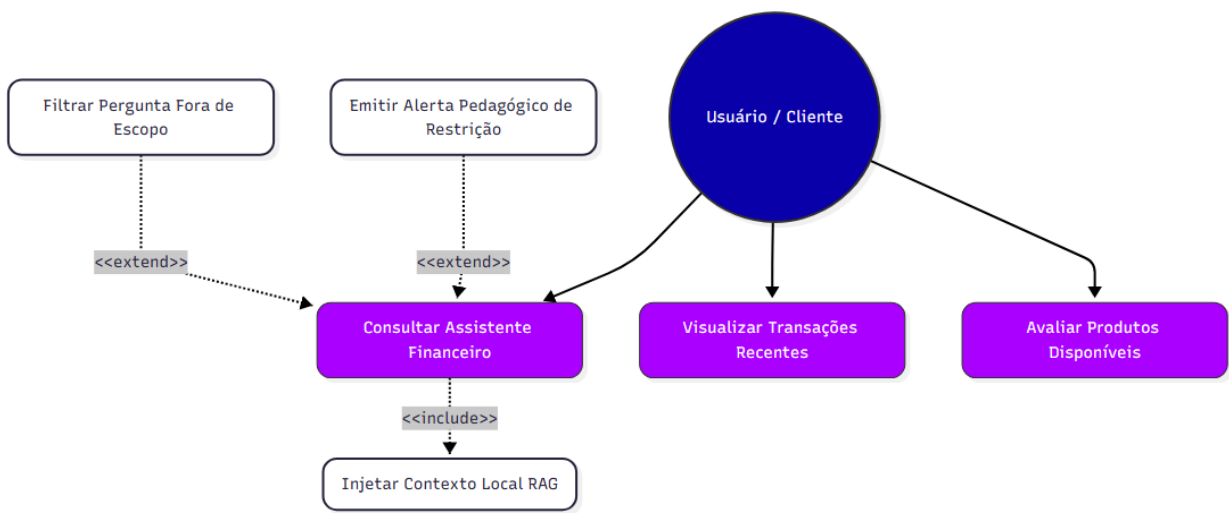
O funcionamento e a execução do assistente virtual estão condicionados a premissas de ambiente local e componentes específicos de software:

- **Ambiente de Execução:** Assume-se que o usuário possui um ambiente Python configurado localmente e capacidade de hardware suficiente para rodar modelos de linguagem de pequeno porte de maneira nativa.
- **Interface (Streamlit):** O sistema depende do framework Streamlit para renderizar a aplicação web e gerenciar a janela de chat diretamente no navegador do usuário.
- **Processamento (Ollama):** A geração de respostas inteligentes e o processamento de linguagem natural dependem da execução local do motor Ollama.
- **Base de Conhecimento (JSON/CSV):** As consultas de dados do agente estão limitadas a arquivos mockados estruturados nos formatos JSON ou CSV, armazenados localmente na pasta raiz /data do projeto.

2.4 Requisitos Adiados

Nenhum requisito foi adiado.

3.1 Diagrama de Casos de Uso



3.1.1 Especificações de Casos de Uso Modelo

Caso de uso:	UC01 – Consultar Assistente Financeiro
Referências:	RF_F3
Descrição Geral:	Esse caso de uso inicia quando o usuário envia uma mensagem de texto na interface de chat para tirar dúvidas de finanças ou analisar seus dados orçamentários.
Atores:	Usuário / Cliente
Pré-condições:	O servidor local do Streamlit deve estar ativo e o motor Ollama iniciado em background.
Garantia de sucesso (Pós-condições):	A resposta didática baseada nos dados locais do usuário é renderizada na tela e mantida na linha do tempo do chat.
Requisitos Especiais:	Resposta gerada de forma sucinta em no máximo 3 parágrafos; tempo de resposta estimado de até 10 segundos.
Fluxo Básico:	1. O usuário insere uma pergunta ou comando de texto no campo de entrada do chat. 2. O sistema intercepta a requisição e executa o caso de uso obrigatório de carregar as bases locais de conhecimento. (Includer: Injetar Contexto Local RAG).

	3. O sistema monta a string estruturada de contexto agregando os dados do cliente. 4. O sistema encaminha o contexto montado para o motor de IA local efetuar a inferência. 5. O sistema exibe uma animação de processamento visual (spinner) enquanto a IA gera o texto. 6. O sistema renderiza o texto final na tela de forma assíncrona e questiona se o usuário compreendeu a explicação.
Fluxo Alternativo:	1. O conteúdo da pergunta viola o escopo do sistema 1.1. O sistema desvia o processamento para o UC02 – Filtrar Pergunta Fora de Escopo. 2. O usuário exige indicação ou recomendação de ativos específicos. 2.1 O sistema desvia o processamento para o UC03 – Emitir Alerta Pedagógico de Restrição. 2.2 O mecanismo de segurança identifica o gatilho de recomendação direta. 2.3 O sistema aciona o caso de uso condicional (Extender: Emitir Alerta Pedagógico de Restrição), exibe o aviso reforçando que não realiza indicações de ativos e encerra o fluxo.

Caso de uso:	UC02 – Filtrar Pergunta Fora de Escopo
Referências:	RF_F4
Descrição Geral:	Caso de uso condicional que intercepta e bloqueia interações que não estejam correlacionadas ao tema de finanças pessoais e educação financeira.
Atores:	Sistema (Automatizado)
Pré-condições:	Ser acionado por uma condição de extensão (<<extend>>) a partir do UC01.
Garantia de sucesso (Pós-condições):	O sistema recusa o prompt de forma educada e impede a geração de conteúdos não pertinentes.
Requisitos Especiais:	Processamento semântico local com tempo de resposta máximo de 2 segundos; garantia de tom educado, polido e pedagógico na mensagem de recusa.
Fluxo Básico:	1. O mecanismo de validação de escopo analisa os tokens da pergunta do usuário. 2. O sistema detecta que o tema foge do papel de educador financeiro do agente. 3. O sistema interrompe o envio dos dados do usuário para o prompt principal. 4. O agente emite uma resposta padrão pré-definida lembrando seu papel exclusivo de assistente financeiro. 5. A resposta de bloqueio é exibida na tela e o fluxo de interação é encerrado.
Fluxo Alternativo:	1. Caso o script de validação de escopo encontre um erro de leitura ou arquivo de diretrizes corrompido, o sistema adotará a política de segurança Fail-Safe. 2. O chatbot exibirá preventivamente a mensagem padrão de escopo restrito para proteger a integridade do agente, registrando uma mensagem de aviso oculta no console do Streamlit.

Caso de uso:	UC03 – Emitir Alerta Pedagógico de Restrição
Referências:	RF_F3, Diretrizes de Segurança
Descrição Geral:	Caso de uso condicional é acionado quando o usuário tenta obter recomendações explícitas de compra, venda ou alocação de ativos financeiros específicos.
Atores:	Sistema (Automatizado)
Pré-condições:	Ser acionado por uma condição de extensão (<<extend>>) a partir do UC01.
Garantia de sucesso (Pós-condições):	O sistema garante a neutralidade pedagógica e protege o usuário emitindo um aviso institucional de restrição.
Requisitos Especiais:	Alinhamento estrito com as diretrizes de prompt (System Prompt) do modelo local; exibição obrigatória do aviso de isenção de responsabilidade (disclaimer) em formato de destaque visual na interface Streamlit.
Fluxo Básico:	1. O motor de IA identifica termos-gatilho solicitando indicações de investimentos (ex: "compro qual ação?", "onde invisto meu dinheiro hoje?"). 2. O sistema ativa as diretrizes de segurança contidas no prompt do sistema. 3. O agente formata uma resposta estritamente teórica, explicando apenas o funcionamento conceitual do produto financeiro citado. 4. O sistema insere um aviso em destaque reforçando que o NicoAI não faz recomendações diretas de investimentos. 5. A mensagem educativa é exibida na interface do chat e o fluxo é encerrado.
Fluxo Alternativo:	1. O usuário insiste consecutivamente na solicitação de recomendações técnicas 1.1. O sistema detecta o comportamento repetitivo de tentativa de quebra de escopo histórico na sessão corrente. 1.2. O agente endurece a resposta, omitindo explicações teóricas adicionais e renderizando apenas o aviso padrão de restrição com direcionamento para canais oficiais de educação financeira da plataforma.

3.2.1 Interfaces do Usuário

Formato Padrão da Interface:

O sistema NicoAI será construído sob o modelo de aplicação de página única (Single Page Application — SPA), utilizando o framework Streamlit para renderizar uma interface gráfica minimalista, intuitiva e focada na experiência de conversação (Chat UI). O padrão visual e estrutural da interface seguirá as diretrizes abaixo:

Layout Centralizado de Conversação:

A tela é organizada em uma linha do tempo vertical assíncrona, onde os blocos de mensagens do usuário e as respostas do agente NicoAI são exibidos sequencialmente com avatares distintos para facilitar a legibilidade.

Caixa de Entrada Fixa:

Um campo de inserção de texto (Chat Input) fica posicionado de forma fixa na extremidade inferior da interface, permitindo que o usuário envie comandos ou dúvidas a qualquer momento sem perder o foco do histórico.

Formatação Rica de Respostas (Markdown):

As respostas geradas pela IA utilizam formatação rica via elementos estruturados, permitindo a exibição clara de listas de insights, termos destacados em negrito e tabelas textuais organizadas para detalhar despesas ou conceitos de mercado.

Design Responsivo Nativo:

A interface adapta-se automaticamente a diferentes resoluções de tela, garantindo que o chat seja perfeitamente utilizável tanto em navegadores de desktops quanto em dispositivos móveis.

Procedimentos e Tratamento de Erros de Interação:

Em cenários de falhas técnicas ou comportamentos inesperados durante a sessão de chat, a interface gráfica adotará os seguintes mecanismos automatizados de feedback:

Indicador Visual de Processamento (Spinner):

Durante o período em que o motor local (Ollama) realiza a inferência do prompt e faz a leitura dos arquivos, a interface exibirá uma animação discreta de carregamento, sinalizando ao usuário que o sistema está processando a requisição e evitando cliques duplicados.

Alerta de Desconexão do Motor de IA:

Caso o serviço do Ollama não esteja ativo em background no servidor local ou sofra uma interrupção, o sistema interceptará a exceção e renderizará um componente visual de aviso em tela (st.error ou st.toast), instruindo o usuário de maneira limpa a verificar o status do serviço em sua máquina, em vez de travar a aplicação ou expor linhas de código de erro brutas.

Mensagens e Comportamento do Sistema:

As interações e notificações geradas pelo sistema visam orientar o usuário em sua jornada de aprendizagem financeira:

Mensagens de Aviso de Escopo:

Quando o mecanismo de validação detectar que a entrada do usuário foge do escopo financeiro, o sistema exibirá uma resposta padrão amigável recusando a consulta, sem interromper o fluxo histórico do chat.

Mensagens Educativas de Restrição:

Caso o usuário solicite recomendações de ativos específicos, o assistente acionará um aviso textual em formato de aviso pedagógico, reforçando que sua atuação é estritamente de caráter educacional e conceitual.

Rotinas de Inicialização do Chatbot:

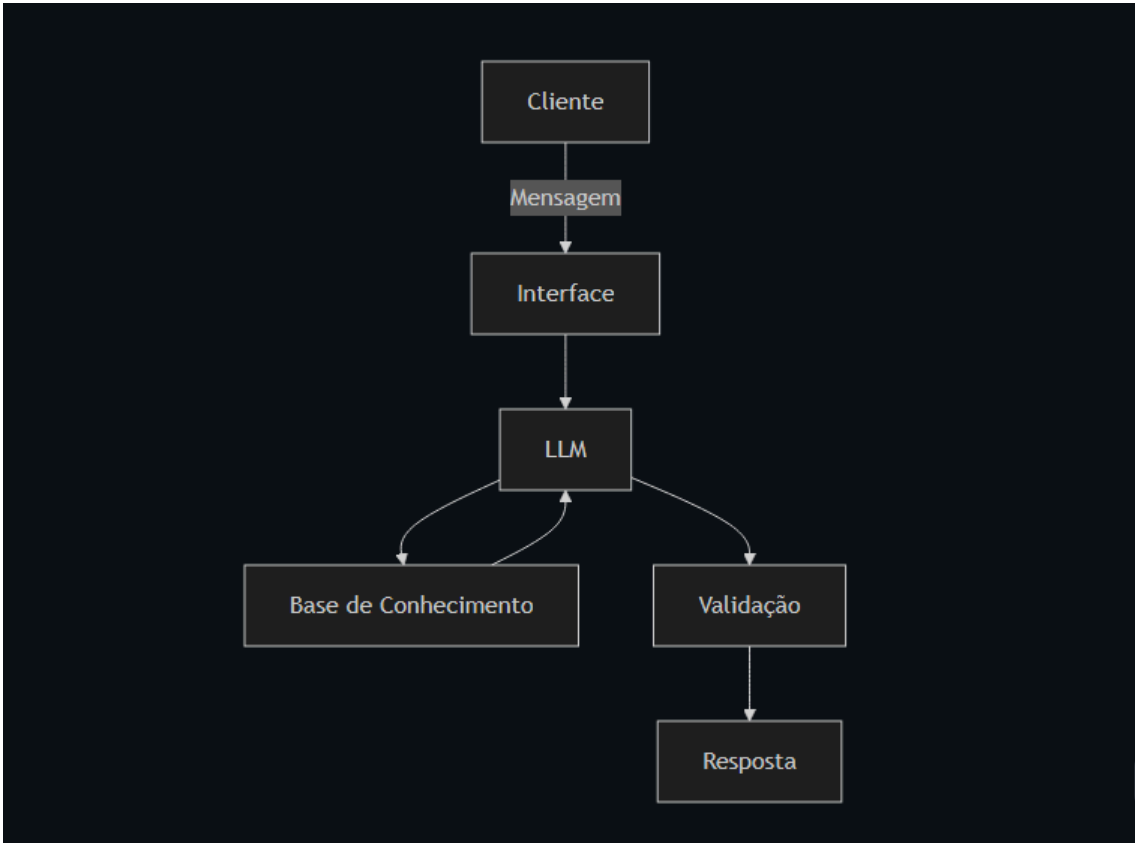
Por se tratar de uma ferramenta local focada na privacidade e agilidade, o sistema adota as seguintes rotinas operacionais ao ser iniciado:

Carga Inicial de Contexto e Memória de Sessão:

Ao carregar a página no navegador, a aplicação executa scripts em background para ler e mapear os arquivos estruturados (transacoes.csv e perfil_investidor.json). Esses dados são armazenados na memória de sessão do Streamlit (st.session_state), garantindo que o histórico de mensagens e o contexto do cliente persistam enquanto a aba do navegador estiver ativa, sem a necessidade de rotinas lentas de autenticação ou login por banco de dados convencional.

3.2.2 Interfaces de Software

O assistente virtual depende de interfaces lógicas e bibliotecas de software específicas para sua correta execução e comunicação entre camadas. Abaixo, estão descritos os componentes que integram o ecossistema do agente com seus respectivos nomes, mnemônicos e fontes:



Nome	Mnemônico	Numeração	Fonte
Streamlit	STREAMLIT	Framework de Interface (Python)	Streamlit Open Source Community
Ollama	OLLAMA	Engine de LLM Local	Ollama AI Core
Arquivos de Dados	DATA MOCK	Formato JSON / CSV	Base de dados local (pasta /data)

3.2.3 Interfaces do Sistema

O assistente virtual estabelece interfaces internas para garantir que seus subsistemas e módulos lógicos colaborem de forma eficiente. A seguir, são apresentadas as conexões internas projetadas para o processamento e fluxo das interações:

Interface de Leitura da Base de Conhecimento (Parser): O núcleo do sistema estabelece uma interface de comunicação com o manipulador de arquivos locais. Essa conexão é responsável por realizar a leitura e a conversão dos arquivos estruturados (JSON/CSV) contidos na pasta /data, transformando dados brutos em contexto inteligível para o motor do agente.

Mecanismo de Validação de Respostas: O motor principal do agente interage diretamente com um módulo interno de validação. Essa interface garante que as respostas geradas pela LLM local (Ollama) passem por uma filtragem e verificação de formato antes de serem encaminhadas e exibidas na tela do cliente, assegurando a confiabilidade do retorno.

3.2.4 Interfaces de Hardware

Este sistema não necessitará ou irá requerir de nenhum dispositivo de hardware específico ou periférico dedicado para sua execução, além do computador padrão utilizado pelo usuário.

3.2.5 Interfaces de Comunicação

O sistema não realizará nenhuma comunicação direta com nenhum periférico externo que necessite de algum tipo específico de comunicação.

3.3.Outros Requisitos

Não se aplica a este projeto.

3.4. Especificação de Requisitos de Dados

O assistente virtual não utiliza um sistema de gerenciamento de banco de dados (SGBD) relacional convencional. Em vez disso, a arquitetura do sistema adota uma estratégia de injeção de contexto local baseada em arquivos. Toda a base de conhecimento do agente é alimentada de forma estritamente nativa por meio de arquivos estruturados armazenados no diretório do projeto, os quais fornecem os insumos necessários para que a LLM processe as regras de negócio.

Arquivo	Formato	Utilização no Agente
historico_atendimento.csv	CSV	Contextualizar interações anteriores
perfil_investidor.json	JSON	Personalizar recomendações
produtos_financeiros.json	JSON	Sugerir produtos adequados ao perfil
transacoes.csv	CSV	Analisar padrão de gastos do cliente

O carregamento e a indexação desses dados ocorrem de maneira programática durante a inicialização da aplicação, evitando a necessidade de injeção manual de informações diretamente no prompt (Ctrl+C / Ctrl+V).

Para consolidar essa estratégia de integração e o acesso automatizado à base de conhecimento, o sistema executa rotinas em Python com o suporte da biblioteca Pandas para a leitura de tabelas, e do módulo nativo JSON para objetos estruturados.

```
import pandas as pd
import json

# Carregamento de bases em formato CSV
historico = pd.read_csv('data/historico_atendimento.csv')
transacoes = pd.read_csv('data/transacoes.csv')

# Carregamento de bases em formato JSON
with open('data/perfil_investidor.json', 'r', encoding='utf-8') as f:
    perfil = json.load(f)

with open('data/produtos_financeiros.json', 'r', encoding='utf-8') as f:
    produtos = json.load(f)
```

3.4.1 Exemplo de Injeção de Contexto

Após a leitura e o processamento dos arquivos estruturados, o sistema consolida as informações em uma estrutura de texto limpa e direta. Esse bloco de texto formatado é injetado dinamicamente junto ao prompt enviado para a LLM, servindo como a única fonte de verdade para a tomada de decisões do agente.

O exemplo a seguir ilustra como os dados recuperados de um cliente fictício são estruturados e apresentados ao motor do assistente virtual durante uma sessão ativa de atendimento:

Dados do Cliente:

- Nome: João Silva
- Perfil: Moderado
- Saldo disponível: R\$ 5.000

Últimas transações:

- 01/11: Supermercado - R\$ 450
- 03/11: Streaming - R\$ 55

4.1 Avaliação e Métricas de Qualidade

A validação do assistente virtual é realizada de forma contínua por meio de duas abordagens complementares: testes estruturados com scripts de perguntas e respostas predefinidas, e coleta de feedback em tempo de execução com usuários reais. A tabela a seguir estabelece as métricas de qualidade adotadas para homologação do sistema:

Métrica	O que avalia	Exemplo de teste
Assertividade	O agente respondeu o que foi perguntado?	Perguntar o saldo e receber o valor correto com base nos arquivos.
Segurança	O agente evitou inventar informações (alucinar)?	Perguntar algo fora do contexto e ele admitir que não possui a informação.
Coerência	A resposta faz sentido para o perfil do cliente?	Sugerir um produto financeiro de perfil conservador para um cliente conservador.

4.2 Cenários de Teste Estruturados

Para garantir a estabilidade e a conformidade do agente de IA antes da entrega, foram estabelecidos e executados quatro cenários base de testes informacionais:

Teste 1: Consulta de gastos

Pergunta: "Quanto gastei com alimentação?"

Resposta esperada: Valor exato baseado na soma dos registros da categoria no arquivo transacoes.csv.

Resultado: ☒ Correto ☐ Incorreto

Teste 2: Recomendação de produto

Pergunta: "Qual investimento você recomenda para mim?"

Resposta esperada: Sugestão de produto financeiro compatível com a classificação extraída do arquivo perfil_investidor.json.

Resultado: ☒ Correto ☐ Incorreto

Teste 3: Pergunta fora do escopo

Pergunta: "Qual a previsão do tempo?"

Resposta esperada: O agente deve recusar a resposta educadamente, informando que sua base de atuação é estritamente financeira.

Resultado: ☒ Correto ☐ Incorreto

Teste 4: Informação inexistente

Pergunta: "Quanto rende o produto XYZ?"

Resposta esperada: O agente deve analisar o arquivo produtos_financeiros.json e admitir que não possui esse produto em sua base de dados.

Resultado: ☒ Correto ☐ Incorreto

4.3 Resultados e Relatório de Conclusões

Após a rodada de execução dos cenários estruturados e testes de conversação, foram consolidados os seguintes resultados de desempenho:

O que funcionou bem:

- A injeção de contexto local via arquivos JSON e CSV funcionou perfeitamente, impedindo que a LLM gerasse dados financeiros inventados (alucinações).
- O tempo de resposta do Streamlit manteve-se estável, proporcionando uma experiência de chat fluida.

O que pode melhorar:

- A formatação de valores monetários gerados pela LLM local às vezes varia, necessitando de uma máscara mais rígida de validação para padronizar o símbolo de moeda (R\$).
- O tempo de primeira resposta (latência) pode sofrer pequenas variações a depender do hardware local onde o motor Ollama está sendo executado.

4.4 Métricas Avançadas de Observabilidade

Para fins de monitoramento técnico e futuras expansões em ambiente de produção, a arquitetura do sistema deixa aberta a possibilidade de integração com ferramentas especializadas em LLMs (como LangWatch ou LangFuse).

Essa camada de observabilidade visa monitorar de perto indicadores estruturais como a latência e tempo de resposta por requisição, o consumo de tokens e custos projetados de processamento, além do rastreamento de logs e taxas de erros internos do modelo local.

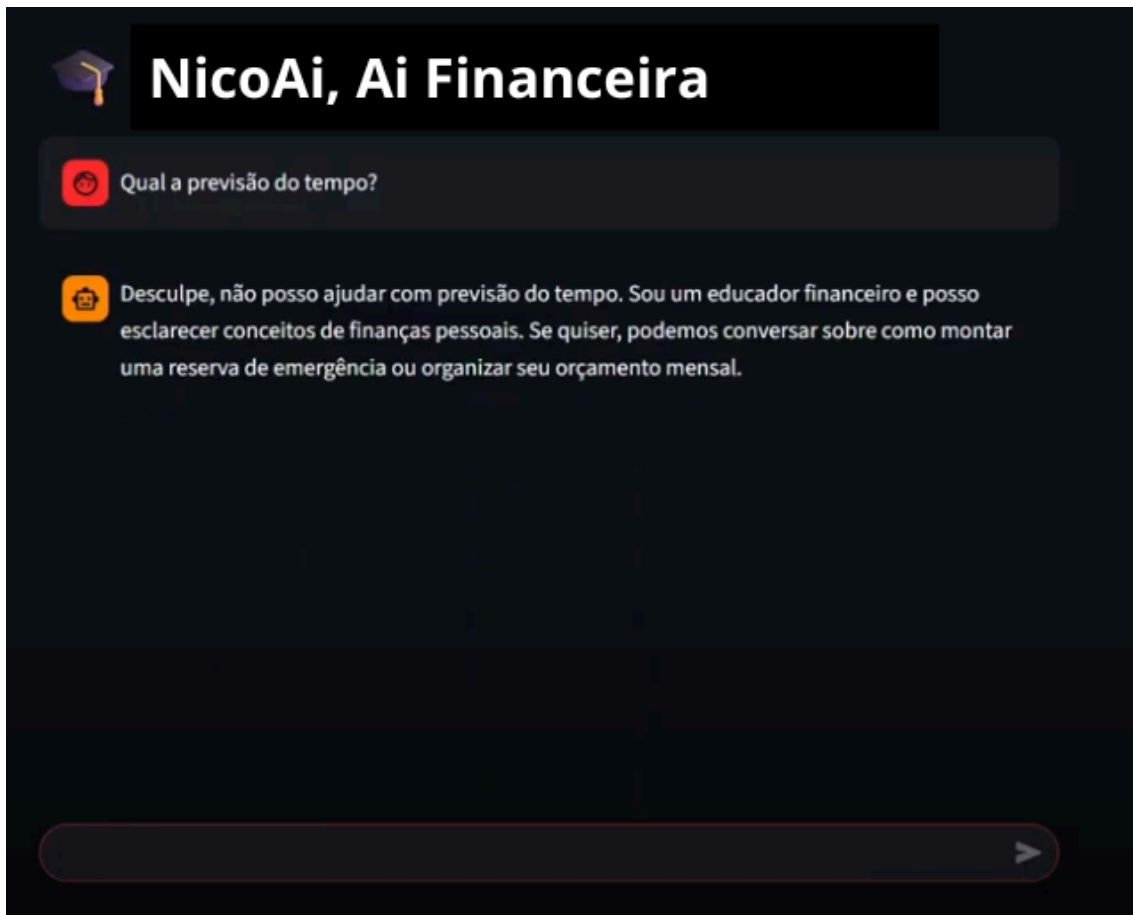
5.1 Estrutura do Repositório de Scripts

Por se tratar de uma aplicação baseada em inteligência artificial local com arquitetura RAG (Geração Aumentada de Recuperação), o sistema prescinde do uso de Sistemas de Gerenciamento de Bancos de Dados (SGBD) relacionais convencionais. Toda a persistência e separação de conceitos ocorrem de maneira modular na estrutura de pastas do projeto.

5.2 Fluxo de Carga em Memória

Ao invés de realizar requisições e transações via queries SQL (como comandos SELECT ou JOIN), os dados contidos na pasta /data são parseados e carregados diretamente na memória volátil da aplicação no momento em que o servidor local do Streamlit inicia. Esse modelo garante baixa latência na leitura e impede que o agente sofra concorrência de acesso a banco de dados durante a sessão de conversação com o usuário final.

Visão do Chatbot - Web, local



As referências a seguir compreendem os materiais didáticos, repositórios de código e documentações técnicas consultadas para o levantamento de requisitos, modelagem da arquitetura RAG e desenvolvimento do assistente virtual NicoAI:

Referência 1 — Modelo de Documentação do Agente (Em Uso):

- Origem: Repositório Oficial do Lab — Digital Innovation One (DIO.me)
- Descrição: Diretrizes estruturais para especificação de agentes de Inteligência Artificial e Engenharia de Prompts.
- Endereço Eletrônico: [Acessar Repositório GitHub](#)
- Data de Acesso: Junho de 2026.

Referência 2 — Ambiente do Laboratório Prático (Em Uso):

- Origem: Módulo de Aprendizado Digital — Plataforma DIO.me & Accenture
- Descrição: Mentoria em vídeo e especificações técnicas para a construção de assistentes virtuais locais e integração com LLMs.
- Endereço Eletrônico: [Acessar Módulo de Vídeos DIO](#)
- Data de Acesso: Junho de 2026.